

DEVELOPMENT OF A STUDENT SKILL-SHARING SYSTEM FOR PEER ASSISTANCE AMONG IT STUDENTS

Presented by:

ACLON, MATTHEO JHON DAMING

DULANAS, FERD DENVER O.

FABRIA, LLOYD VINCENT FORDAN

REBUYON, ERVEN DENOSTA

SENARLO, CLYDE OWEN OLAPE

FAB-BSIT2-10

OCTOBER 2026

ABSTRACT

Information Technology (IT) students need to learn many technical skills, such as programming, database management, web development, and networking. Every student has different strengths, so a student who is good at Java may still need help with databases. Right now, students usually look for help through personal messages and group chats. Requests are easy to lose, and it is hard to know which classmate really has the skill that is needed.

This project presents a Student Skill-Sharing System for Peer Assistance Among IT Students. It is a desktop application where students can list their skills, search for classmates who have a certain skill, send a help request, and join a lobby for group learning. After the help is done, the requester can rate the helper and leave a comment. The system uses Java Swing for the screens and a MySQL database with seven tables: Student, Skills, UserSkill, Request, Lobby, LobbyMember, and Feedback.

The project follows an Agile SDLC supported by Scrum practices. This document explains the background, objectives, scope, feasibility, requirements, process flow, database design, methodology, risks, schedule, budget, and Scrum plan of the project. Screenshots of the working system are included.

Keywords: student skill-sharing, peer assistance, information technology, collaborative learning, database system

1. PROJECT BACKGROUND

1.1 Client / Organization Background

The Student Skill-Sharing System is made for IT students. In this project, the students themselves are the clients and the main users. A student can be a **requester** (someone who needs help) or a **helper** (someone who has a skill and is willing to share it), and the same student can be both at different times. Instructors are indirect beneficiaries because the system supports learning outside the classroom, but it does not replace formal instruction. The system is a desktop application connected to one shared database.

1.2 Current Process

At present, a student who is stuck on a topic usually asks a friend in person or sends a message in a group chat. Classmates who see the message may or may not answer, and the student often does not know who is good at that topic. When someone agrees to help, the meetup is arranged through chat, and nobody keeps a record of the help that was given or whether it was useful.

1.3 Identified Problems

Based on this process, the following problems were identified:

- Students have difficulty finding classmates who have the skills they need.
- There is no single place where students can list their IT skills and where others can look for them.
- Help requests sent through chats are easy to miss and hard to track.

- Students have no organized way to gather for group learning when several of them need help with the same topic.
- There is no consistent place to record feedback about peer help, so good helpers are not recognized.

2. PROJECT PURPOSE AND OBJECTIVES

2.1 Project Purpose

The purpose of the project is to give IT students one organized place to share what they know and ask for help when they need it. Instead of scrolling through group chats, a student can search for a skill, see who has it, check the helper's rating, and send a request in a few clicks.

2.2 Main Project Objective

To design and develop a desktop-based Student Skill-Sharing System that helps IT students share skills, request assistance, and connect with classmates for individual or group-based learning.

2.3 Specific Objectives

- To let students create an account, log in, and manage their profile and skills.
- To let students search for classmates by skill and see their year level and rating.
- To let students send help requests and monitor the status of each request.
- To let helpers accept or decline requests, send notes, and mark requests as done.
- To provide lobbies where a helper can gather accepted students and set the meetup date and time.
- To let requesters rate their helpers and let students view the comments they received.

- To store all student, skill, request, lobby, and feedback records in an organized database.

3. PROJECT SCOPE AND LIMITATIONS

3.1 Project Scope

The desktop application includes the following major functions:

User / Area	Included Functions
Student Account	Register with student ID, school year, name, username, and password. Log in, log out, and edit the profile.
Skills Profile	Add skills by ticking Java, Python, C++, JavaScript, HTML, or CSS, or by typing other skills. Remove skills after confirming. View own rating and comments.
Requester	Search for students by keyword and skill filter, send a help request, check My Request, view notes from the helper, and rate the helper.
Helper	View requests from other students, accept or decline, send notes, create and open a lobby, and mark requests as Done.
Lobby	Show the skill, notes, meetup date and time, and participants. Send notes, date, and time to all, mark all as done, and close.
System / Database	Store data in seven tables and keep the request status (Pending, Accepted, Declined, Done) and the average rating of each student.

3.2 Project Limitations

- The system is for IT students only and runs as a desktop application. There is no web or mobile version.
- There is no chat. Helpers and requesters communicate through notes, and meetup details are shared through the lobby.
- Matching is manual. The system lists students who have the skill, but the requester chooses who to ask. Listing a skill does not guarantee that the student is available or an expert.
- There are no email or push notifications, and there is no administrator screen for managing skills.
- The system does not replace instructors, tutors, or formal lessons.

4. PROJECT FEASIBILITY AND PROJECT CHARTER SUMMARY

4.1 Technical Feasibility

The project is technically feasible for a student team. It is built as a Java desktop application with a Swing interface, and its records are kept in a MySQL database made up of seven tables. Java and database design are already part of our IT coursework, so the team does not need to learn a new platform. The tools are free and run on an ordinary school or home computer.

4.2 Operational Feasibility

The system is operationally feasible because its screens are simple and follow one pattern. After logging in, a student sees a main menu with three buttons: Profile, Find Skills, and Request. Every other screen has a Back button, and short messages such as "Request sent!" and "Skill

removed!" tell the student what happened. Since the users are IT students who already use computers every day, very little training is needed.

4.3 Economic Feasibility

The project is economically feasible because the development tools are free and the team provides the labor as part of a school requirement. The only expected costs are internet load, printing, transportation for testing, and a USB drive for backups. These are small compared with the benefit of giving students one organized place to find help.

4.4 Project Charter Summary

Charter Item	Entry
Project Title	Development of a Student Skill-Sharing System for Peer Assistance Among IT Students
Project Purpose	Give IT students one organized place to list their skills, ask for help, and find classmates who can assist them.
Primary Users	IT students who need help and IT students who are willing to help.
Major Deliverable	A working desktop application with a database, together with complete documentation.
Main Constraint	Limited school term, student-level resources, and the need to test with real classmates.
Project Success	A student can register, add skills, find a helper, send and answer requests, meet in a lobby, and rate the help received.

5. REQUIREMENTS GATHERING AND ANALYSIS

5.1 Stakeholders

Stakeholder	Role / Interest
IT Students (Requesters)	Search for classmates with a needed skill and send requests for help.
Student Helpers	List their skills, accept or decline requests, and hold meetups with the students they help.
Instructors	May encourage the use of the system as an extra support tool for peer learning.
Development Team	Plans, designs, builds, tests, and documents the system.

5.2 Requirements Gathering Methods

- Observation of how classmates ask for help through personal messages and group chats.
- Informal talks with classmates about the skills they usually need help with, such as Java, Python, C++, JavaScript, HTML, and CSS.
- Review of the problems in Section 1.3 so that every problem has a matching feature.
- Testing of the working screens by the team, followed by corrections after each round.

5.3 Functional Requirements

ID	Functional Requirement
FR-01	The system shall allow a new student to create an account using student ID, school year, first name, last name, username, password, and password confirmation.
FR-02	The system shall allow a registered student to log in and shall show an error when the details are wrong.
FR-03	The system shall show a main menu with Profile, Find Skills, Request, and Logout options.
FR-04	The system shall show and allow editing of the student's profile, including the rating and the list of skills.
FR-05	The system shall allow a student to add skills from a list or by typing other skills, and to remove a skill after confirming.
FR-06	The system shall allow a student to search for classmates by keyword and skill filter and show their skills, student ID, year level, and rating.
FR-07	The system shall allow a student to send a help request to a selected classmate with the starting status Pending.
FR-08	The system shall let a requester view his or her requests, filter them, view notes from the helper, and rate the helper.
FR-09	The system shall let a helper view requests from other students and accept, decline, or send notes about a request.

ID	Functional Requirement
FR-10	The system shall let a helper create and open a lobby for accepted students, send notes, meetup date, and time to all members, and mark all members as done.
FR-11	The system shall let a helper mark a request as Done once the help is finished.
FR-12	The system shall show the average rating of a student on the profile, or "No ratings yet", and let the student view comments.
FR-13	The system shall store all records in the database.

5.4 Non-Functional Requirements

Category	Requirement
Usability	Screens should use clear labels and buttons, and every screen should have a way to go back.
Performance	Searching, sending a request, and opening a lobby should respond within a few seconds.
Security	A student may only manage his or her own profile and requests.
Reliability	Saved requests, skills, and lobby details must still be there after the program is closed and opened again.
Data Integrity	Records are linked by primary and foreign keys, and a student must not list the same skill twice.

Category	Requirement
Maintainability	Skills and year levels should be easy to update without rewriting the whole program.

5.5 User Stories and Acceptance Criteria

User Story	Acceptance Criteria
As a student, I want to list my skills so that classmates know what I can help with.	After clicking Add Skill and OK, the chosen skills appear in My Skills.
As a student, I want to search for a skill so that I can find someone who can help me.	The table shows matching students with their year level and rating.
As a student, I want to send a request so that I do not need group chats.	"Request sent!" appears and the request shows as Pending in My Request.
As a helper, I want to accept requests and open a lobby so that I can help several classmates at once.	The status changes to Accepted, and Create Lobby groups the accepted students in one window.
As a student, I want to rate my helper so that others know who gives good help.	After Rate Helper, the helper's rating is updated.

6. DEVELOPMENT METHODOLOGY, SDLC MODEL, AND SOFTWARE TOOLS

6.1 Development Methodology Used

The team uses the Agile methodology supported by the Scrum framework. The work is divided into short development cycles called sprints. In each sprint, the team picks the most important features from the product backlog, builds and tests them, shows the result, and uses the comments to plan the next sprint. This keeps the project moving in small and visible steps instead of one long build with a single deadline.

6.2 SDLC Model Used

The selected SDLC model is the Agile SDLC. Instead of finishing all the planning, then all the design, then all the coding, the team repeats the same stages in every sprint: planning, requirements analysis, design, development, testing, and review. This fits the project because the features depend on each other in a clear order: accounts first, then skills, then requests, then lobbies and feedback.

6.3 SDLC Model Figure



Figure 5. Agile SDLC Model Used for the Project

6.4 Software and Development Tools Used

Software / Technology	Use in the Project
Java	Main programming language used to write the application's logic.
Java Swing	Used to build the screens: login, register, main menu, profile, search, requests, and lobby.
MySQL	Relational database that stores the seven tables.
draw.io	Used to draw the process flowchart and the ERD.
Java IDE (such as NetBeans or Eclipse)	Used to write, run, and debug the program.

Software / Technology	Use in the Project
GitHub	Used to back up project files and share work among team members.

6.5 Reason for Selecting the Methodology and SDLC Model

Agile with Scrum was chosen because the system is made of features that can be finished and tested one at a time. A student can already log in and edit a profile even before the lobby feature exists. Short sprints also help the team handle a limited school term. If a part is not finished, it moves to the next sprint without stopping the whole project, and testing after every sprint makes errors easier to find.

6.6 Expected Development Approach

- Prepare and prioritize the system requirements.
- Divide the work into small backlog items and build a working set in every sprint.
- Test finished features before adding more work.
- Show progress to classmates or the instructor and improve the system with their feedback.

7. SDLC STAGES AND PROJECT APPLICATION

SDLC Stage	Application to the Student Skill-Sharing System
Planning	Identify the problem of finding classmates who can help, set the objectives and scope, and choose the tools and schedule.
Requirements Analysis	List what students need: accounts, skill profiles, search, requests, lobbies, and ratings.
System Design	Design the seven database tables, draw the process flowchart, and plan the layout of each screen.
Development	Code the Swing screens and connect them to the database, one group of features per sprint.
Testing	Test registration, login, skills, search, requests, lobbies, and ratings, including wrong inputs.
Deployment	Install the application on the computers of the students who will test and use it.
Maintenance	Fix errors found by users, update the skill list, and add improvements such as notifications or automatic matching.

8. PROJECT RISK MANAGEMENT

8.1 Possible Risks Discovered

The team listed the events that could affect the schedule, scope, or quality of the project and prepared a response for each one.

8.2 Risk Assessment and Response

Risk	Probability	Impact	Response / Mitigation
Not enough students list their skills, so search results look empty.	Medium	High	Ask classmates to register during testing and add sample skills for the demonstration.
Students list skills they do not really have.	Medium	Medium	Show helper ratings and comments so that other students can judge the quality of the help.
The database and the program do not match.	Medium	High	Check every table and field against the ERD at the end of each sprint.
The project takes longer than the school term allows.	Medium	High	Build the core features first (accounts, skills, requests) and move extra features to a later sprint.
Project files or the database are lost.	Low	High	Keep the code on GitHub and export a database backup every sprint.

Risk	Probability	Impact	Response / Mitigation
A team member becomes unavailable.	Medium	Medium	Share all files, write short notes on each module, and assign a backup member.
Passwords or personal details are exposed.	Low	High	Limit database access, show only the information needed for helping, and plan to protect stored passwords.

9. PROJECT TIME MANAGEMENT AND SCHEDULE

9.1 Major Project Activities

The schedule follows a twelve-week school term and can be adjusted to the actual academic calendar.

9.2 Project Schedule

Activity	Duration	Target Period	Main Output
Planning and Problem Study	1 week	Week 1	Problem, objectives, scope
Requirements and Design	2 weeks	Weeks 2–3	Requirements, flowchart, database design

Activity	Duration	Target Period	Main Output
Sprint 1 Development	2 weeks	Weeks 4–5	Register, login, main menu, profile, skills
Sprint 2 Development	2 weeks	Weeks 6–7	Search skills, request help, My Request, accept and decline
Sprint 3 Development	2 weeks	Weeks 8–9	Lobbies, notes, date and time, done, ratings
Testing and Revision	2 weeks	Weeks 10–11	Tested and corrected version
Final Documentation and Presentation	1 week	Week 12	Final paper and demonstration

9.3 Project Milestones

Milestone	Target
Requirements and database design approved	End of Week 3
Accounts and skill profiles working	End of Week 5
Requests and helper responses working	End of Week 7
Lobbies and ratings working	End of Week 9

Milestone	Target
Testing finished and errors fixed	End of Week 11
Final presentation	End of Week 12

10. PROJECT BUDGET AND COSTING

10.1 Estimated Project Cost

Cost Item	Basis	Estimated Cost
Development Software	Java, IDE, MySQL, and draw.io are free	PHP 0
Internet Allowance	Research, online meetings, and GitHub	PHP 1,000
Transportation	Meetings and testing with classmates	PHP 500
Printing and Documentation	Drafts and final copies	PHP 800
Backup Storage	USB drive for project and database backup	PHP 500
Contingency	Allowance for unexpected expenses	PHP 700
TOTAL ESTIMATED COST		**PHP 3,500**

10.2 Cost Justification

The budget only covers expenses the team will actually pay for and does not charge for student labor. Internet and transportation support meetings and testing, printing is needed for the drafts and final copies, and backup storage protects the code and the database. The amounts are estimates and will be updated with the real costs.

11. SCRUM / AGILE APPLICATION

11.1 Scrum Roles

Scrum Role	Responsibility
Product Owner	Represents the needs of the students who will use the system and decides which features come first.
Scrum Master	Organizes the sprint activities, helps remove problems, and keeps the team focused.
Development Team	Designs, codes, tests, and documents the system.

11.2 Product Backlog

Priority	Backlog Item	Expected Result
High	Registration and login	Students can create an account and log in.
High	Profile and skills	Students can edit their profile and add or remove skills.

Priority	Backlog Item	Expected Result
High	Search and request help	Students can find classmates by skill and send a request saved as Pending.
High	Accept and decline	Helpers can respond to requests, and the status is updated.
Medium	Lobby	Helpers can group accepted students and send notes, date, and time to all.
Medium	Rating and comments	Students can rate a helper, and the rating shows on the profile.
Low	Extra features	Notifications, messaging, and automatic matching if time allows.

11.3 Sprint Plan

Sprint	Sprint Goal	Main Work
Sprint 1	Set up accounts and profiles.	Database setup, register, login, main menu, profile, add and remove skills.
Sprint 2	Connect students who need help with those who can help.	Search skills, request help, My Request, accept, decline, send notes.
Sprint 3	Support group learning and feedback.	Lobbies, meetup details, send notes to all, mark done, rate helper, view comments.

Sprint	Sprint Goal	Main Work
Sprint 4	Prepare the system for use.	Testing, bug fixing, user feedback, and final documentation.

11.4 Scrum Events

Scrum Event	How the Team Applies It
Sprint Planning	Choose the backlog items the team can finish and set the sprint goal.
Daily Scrum / Check-in	A short check-in in person or through a group chat: what was finished, what is next, and what is blocking.
Sprint Review	Show the finished features to classmates or the instructor and write down their comments.
Sprint Retrospective	Discuss what went well, what caused delays, and what to improve.

11.5 Simple Scrum Progress Tracking

The team uses a simple task board with three columns: To Do, In Progress, and Done. Each backlog item is given to a member and moved across the board as the work moves forward.

12. PROJECT SUMMARY

IT students need many technical skills, and every student is stronger in some areas than in others. The problem is that classmates do not know who can help with a particular topic, and

requests sent through chats are easy to lose. The Student Skill-Sharing System answers this with one organized system. Students list their skills, search for classmates by skill, send requests, and follow the status of each request. Helpers accept or decline, gather students in a lobby with notes, date, and time, and mark the session as done. Students then rate their helper, so the next student knows who to ask. All of this is kept in a database of seven connected tables.

The feasibility review shows that the project is realistic: the tools are free and familiar to the team, the screens are simple, and the costs are small. Agile and Scrum let the team build the system in short sprints and test each group of features before adding the next. The flowchart, ERD, and screenshots show that the main features already work as described. The real benefit will depend on how many students use the system and how willing they are to help one another, so the next steps are testing with more IT students and adding improvements such as notifications, messaging, and stronger password protection.